

## CHAPTER : 8

### ★ "DOUBLE ENDED QUEUE (DEQUE)" ★

#### DEQUE :-

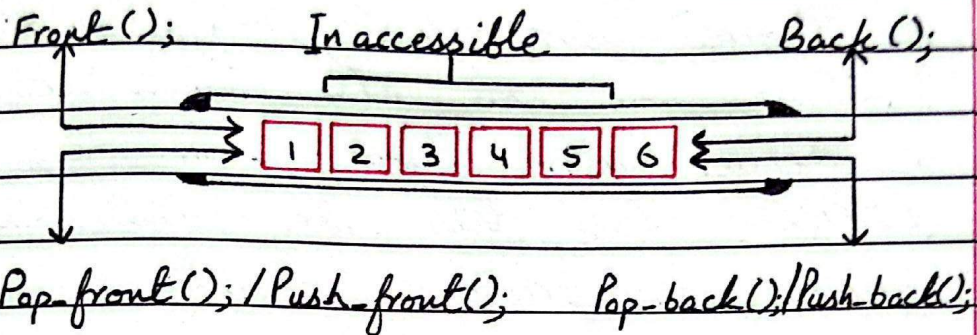
##### Definition -

"Deque is a type of data structure formed by fusion of a stack and queue, it allows insertion and deletion from both ends like a list, but the data in between these ends cannot be accessed directly."

##### Explanation -

Consider a list, you can enter and delete data from both, front and back. However, you have a `searchNode()` in list which allows list traversal, similarly `deleteNode` and `insertNode`. Such methods are not allowed in a deque, you only have direct access to only front and back.

## MEMORY DIAGRAM:-



With that, you can say that a deque is not much different than a list with restrictions.

## DEQUE USING LIST:-

Lets implement a deque using list defined in C++ STL. Its code is quite simple.

Code:-

```
class deque {
    std::list<int> l;
public:
    void push-back(int n) { l.push-back(n); }
    void push-front(int n) { l.push-front(n); }
    void pop-back() { l.pop-back(); }
    void pop-front() { l.pop-front(); }
    int back() { return l.back(); }
    int front() { return l.front(); }
    bool empty() { return l.empty(); }
};
```

Date: \_\_\_/\_\_\_/\_\_\_

Day \_\_\_

This is how we can implement a deque using list from C++ STL. Similarly you can also implement a deque using your own ListNode class like we did with queue in previous chapter.

Apart from this, there isn't much about deque. Deque is just a special queue that allows manipulation at both ends.

This is all about deque for now. Next up we will dive into strings. As always, the deque's code is present in the github repo along with its scratch implementation using our own ListNode class.

["github.com/coder-Retro/Fast-University"](https://github.com/coder-Retro/Fast-University)

Go into DSA > Deque.

Next is strings, Till Then Good Luck!